

图论基础

李子涵

西安交通大学

2026 年 7 月 14 日

目录 I

- ① 图
- ② 树
- ③ 图的存储
- ④ 图上搜索
- ⑤ 拓扑排序
- ⑥ 基础树上问题
- ⑦ 最小生成树
- ⑧ 最短路
- ⑨ 最近公共祖先
- ⑩ 结语



图是一个二元组 $G = (V(G), E(G))$.

其中 $V(G)$ 是非空集合，称为点集。 V 中的每个元素被称为顶点或节点，简称点。

$E(G)$ 为 $V(G)$ 各节点之间边的集合，称为边集。

常用 $G = (V, E)$ 表示图。

方向性

若 G 为无向图，则 E 中的每个元素为一个无序二元组 (u, v) ，称作无向边，简称边，其中 $u, v \in V$ 。设 $e = (u, v)$ ，则 u 和 v 称为 e 的端点。

若 G 为有向图，则 E 中的每个元素为一个有序二元组 (u, v) ，有时也写作 $u \rightarrow v$ ，称作有向边或弧，在不引起混淆的情况下也可以称作边。设 $e = u \rightarrow v$ ，则此时 u 是 e 的起点， v 是 e 的终点，起点和终点也可称为 e 的端点。并称 u 是 v 的直接前驱， v 是 u 的直接后继。

边

若 G 的每条边 $e_k = (u_k, v_k)$ 都被赋予一个数作为该边的权, 则称 G 为赋权图。如果这些权都是正实数, 就称 G 为正权图。
在无向图 $G = (V, E)$ 中, 若点 v 是边 e 的一个端点, 则称 v 和 e 是关联的或相邻的。对于两顶点 u 和 v , 若存在边 (u, v) , 则称 u 和 v 是相邻的。

简单图

对于 E 中的边 $e = (u, v)$, 若 $u = v$, 则 e 被称作一个自环。
若 E 中存在两条完全相同的边 e_1, e_2 , 则它们被称作（一组）重边。

若一个图中不存在重边与自环, 则它被称为简单图。

度数

与一个顶点 v 关联的边的条数称作该顶点的度, 记作 $d(v)$ 。特别的, 每条边 (v, v) 会对 $d(v)$ 产生 2 的贡献。

在有向图中 $G = (V, E)$ 中, 以一个顶点 v 为起点的边的条数称为该顶点的出度, 记作 $d^+(v)$ 。以一个顶点 v 为终点的边的条数称为该顶点的入度, 记作 $d^-(v)$ 。显然 $d^+(v) + d^-(v) = d(v)$ 。

路径

途径是连接一连串顶点的边的序列。一条有限途径 w 是一个边的序列 $e_1, e_2, \dots, e_k$, 使得存在一个顶点序列 $v_0, v_1, \dots, v_k$ 满足 $e_i = (v_{i-1}, v_i)$, 其中 $i \in [1, k]$ 。这样的途径可以简写为 $v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ 。通常定义途径的长度为边的数量 k 。对于一条途径 w , 若 $e_1, e_2, \dots, e_k$ 两两互不相同, 则称 w 是一条迹。

对于一条迹 w , 若其连接的点的序列中的点两两不同, 则称 w 是一条路径, 又称简单路径。

对于一条迹 w , 若 $v_0 = v_k$, 则称 w 是一条回路。

对于一条回路 w , 若 $v_0 = v_k$ 是点序列中唯一重复出现的点对, 则称 w 是一个环。

子图

对于一张图 $G = (V, E)$, 若存在另一张图 $H = (V', E')$ 满足 $V' \subseteq V$ 且 $E' \subseteq E$, 则称 H 是 G 的子图, 记作 $H \subseteq G$ 。

若对 $H \subseteq G$, 满足 $\forall u, v \in V'$, 只要 $(u, v) \in E$, 均有 $(u, v) \in E'$, 则称 H 是 G 的导出子图。点集为 V' ($V' \subseteq V$) 的导出子图称为 V' 导出的子图, 记作 $G[V']$ 。

若 $H \subseteq G$ 满足 $V' = V$, 则称 H 为 G 的生成子图。

连通性

对于一张无向图 $G = (V, E)$, 对于 $u, v \in V$, 若存在一条途径使得 $v_0 = u, v_k = v$, 则称 u 和 v 是连通的。

若无向图 $G = (V, E)$, 满足其中任意两个节点均连通, 则称 G 是连通图, G 的这一性质称作连通性。

若 H 是无向图 G 的一个连通子图, 且不存在 F 满足 $H \subsetneq F \subseteq G$ 且 F 为连通图, 则 H 是 G 的一个连通块, 又称连通分量。

对于一张有向图 $G = (V, E)$, 对于 $u, v \in V$, 若存在一条途径使得 $v_0 = u, v_k = v$, 则称 u 可达 v 。

特殊的图

若无向简单图 G 满足任意不同两点间均有边, 则称 G 为完全图。
若有向简单图 G 满足任意不同两点间均恰有两条方向不同的边, 则称 G 为有向完全图。

如果一张无向连通图不含环, 则称它是一棵树。多棵树可以组成一个森林。

如果一张无向连通图包含恰好一个环, 则称它是一棵基环树。

如果一张有向图不含环, 则称它是一张有向无环图 (DAG)。

树是不含环的无向连通图。由于这点，树具有更多的特殊性质，因此单独讨论。

树

一个没有固定根结点的树称为无根树。

在无根树的基础上，指定一个节点称为根，则形成一棵有根树。

生成树是一个连通无向图的生成子图，同时要求是树。

无根树上度数不超过 1 的节点被称为叶结点。

有根树

对于除根结点以外的每个结点，定义从该结点到根结点路径上的第二个结点为父结点。根节点没有父结点。

一个结点到根结点的路径上除自身以外的所有结点均为该结点的祖先。根节点的祖先集合为空。

如果 u 是 v 的父亲，那么 v 是 u 的子结点。没有子结点的结点为叶节点。

一个结点的深度为其到根结点的路径上的边数，一棵树的高度为所有结点深度的最大值。

有根树

同一个父亲的多个子结点互为兄弟。

一个点的后代包含其子结点和子结点的后代。若 u 是 v 的祖先，那么 v 是 u 的后代。

一个结点的子树为其和所有后代构成的集合的导出子图。

特殊的树

满足对于任一结点，相连的边不超过 2 条的树称为链。

如果一棵树满足存在一个结点 u ，使得所有除了 u 以外的结点均与 u 相连，则这棵树被称为菊花。

每个节点最多只有两个子结点的有根树称为有根二叉树。常常对两个子结点的顺序加以区分，分别称之为左子节点和右子节点。

如果一棵二叉树只有最下方两层的度数小于 2，且最下方一层的结点都集中在该层最左边的连续位置上，则被称为完全二叉树。

如果一棵二叉树的所有叶结点的深度相同，且所有非叶节点的子节点数量均为 2，则被称为完美二叉树。

在算法竞赛中，想要对图进行操作，就需要先学习图的存储方式。接下来对各种的存边方式的复杂度分析中，用 n 表示图的点数，用 m 表示图的边数。

以下的讨论均针对有向图。对于无向图，可以将每条边视作两条方向相反的边后通过有向图的存边方式存边。

直接存边 - 方法

直接使用一个数组存边，数组中的每个元素都包含一条边的起点和终点。对于带边权的图同时存储其边权。

用多个数组分别存储每条边的起点、终点和边权也属于这种方式。

直接存边 - 复杂度

查询某条边是否存在: $O(m)$ 。

遍历一个点的所有出边: $O(m)$ 。

遍历整张图: $O(nm)$ 。

空间复杂度: $O(m)$ 。

直接存边 - 应用

一般不用于遍历图。

当需要将所有边按边权排序时，可以先通过该方法存图以排序。
对于一些需要多次建图的题目，可以通过直接存图把边存下来，之后再通过存下来的边建图。

邻接矩阵 - 方法

使用一个二维数组 $\text{adj}[][]$ 存边。具体的，当存在 u 到 v 的边时， $\text{adj}[u][v]$ 为 1，否则为 0。

对于赋权图，可以在 $\text{adj}[u][v]$ 中存储 u 到 v 的边权。

邻接矩阵 - 复杂度

查询某条边是否存在： $O(1)$ 。

遍历一个点的所有出边： $O(n)$ 。

遍历整张图： $O(n^2)$ 。

空间复杂度： $O(n^2)$ 。

邻接矩阵 - 应用

可以非常高效的判断一条边是否存在。

一般只在稠密图上使用。

对于重边，需要判断重边能否忽略。如果不能忽略，则邻接矩阵不适用。

邻接表 - 方法

对于每个点，使用一个支持动态增加元素的数据结构存储它的所有出边的相关信息，如终点、边权等。

实际应用中，对于 C++，往往使用 `vector<int> adj[]` 存边，其中 `vector<int> adj[u]` 中存储了点 u 的出边的信息。

邻接表 - 复杂度

查询某条边是否存在： $O(d^+(u))$ 。

遍历一个点的所有出边： $O(d^+(u))$ 。

遍历整张图： $O(n + m)$ 。

空间复杂度： $O(m)$ 。

邻接表 - 应用

可以应对大多数的场景。

特别的优势在于使用邻接表可以对一个点的所有出边进行排序。

链式前向星 - 方法

本质上是使用链表实现的邻接表。主要特征是将所有点出边的链表压进了一个数组中。

每条边除了终点、边权等信息外，还需记录其在链表中指向的下一个元素。同时，每个点需要记录链表头的索引。

链式前向星 - 复杂度

查询某条边是否存在： $O(d^+(u))$ 。

遍历一个点的所有出边： $O(d^+(u))$ 。

遍历整张图： $O(n + m)$ 。

空间复杂度： $O(m)$ 。

链式前向星 - 应用

可以应对大多数的场景。

理论上常数相比邻接表更加优秀，但是不能快速查询一条边是否存在或对边进行排序。

搜索是大量图上算法的重要基础。

图上的基础搜索方式有宽度优先搜索 (BFS) 和深度优先搜索 (DFS)。两者的大致思想与在搜索算法中是类似的。

BFS - 原理

BFS (Breadth First Search), 中文名是宽度优先搜索。

该算法中, 每次尝试访问同一层的结点, 如果都访问完了, 就访问下一层。

通过该算法, 每个节点都是通过从起点到该点的边数最少的路径访问的。

BFS - 实现

与搜索算法中的 BFS 类似的，用一个队列 Q 来记录要处理的结点，然后用一个布尔数组 $\text{vis}[]$ 来标记是否已经访问过某个结点。

算法开始时，将所有节点的 vis 值设为 false ，表示未访问，然后把起点 s 放入队列 Q 并将 $\text{vis}[s]$ 的值设为 true 。

接着每次从队列 Q 中取出队首的结点 u ，然后把与 u 相邻的所有节点 v 的 vis 值设为 true 并放入队列 Q 。这个操作被称为“扩展”

重复上述操作直至 Q 为空，表示 BFS 结束。

BFS - 实现

考虑找出一条具体的最短路径。

记录一个数组 `pre`，用以记录每个点在抵达它的最短路径上的前驱。在每次从 u 扩展到 v 时，将 `pre[v]` 设为 u 。特别的，起点 s 满足 `s` 为某个空值，例如 `-1`。

在搜索结束后，根据前驱回溯并倒序输出，即可得到最短路径。如果让每个点和自己的前驱连边，实际上构成了一棵以 s 为根的生成树，满足每个点到根节点的距离都等于原图上每个点到起点的距离。

BFS - 应用

无权图最短路

给定一张无权图和源点 s ，求 s 到其它所有点的最短路的长度。

BFS - 应用

无权图最短路

给定一张无权图和源点 s ，求 s 到其它所有点的最短路的长度。

从 s 开始 BFS。

在 BFS 过程中记录数组 dis ，用以记录 s 到每个点的最短路长度。初始时 $\text{dis}[s]$ 为 0。

当从 u 拓展到 v 时，将 $\text{dis}[v]$ 设为 $\text{dis}[u] + 1$ 。

也可以考虑使用原先的 vis 数组实现这个功能。

队列里元素的 dis 总是递增且差不超过 1 的。

BFS - 应用

找最短路上的点

给定一张无权图和起点 s 以及终点 t ，找出可能在 s 到 t 的最短路上的点。

从 s 和 t 分别进行一次 BFS，记录距离分别为 ds 和 dt 。

对于一个点 u ，如果 $ds[u] + dt[u]$ 等于 $ds[t]$ ，则说明必定存在经过 u 的最短路，否则一定不存在。

BFS - 应用

找最短路上的边

给定一张无权图和起点 s 以及终点 t ，找出可能在 s 到 t 的最短路上的边。

BFS - 应用

找最短路上的边

给定一张无权图和起点 s 以及终点 t ，找出可能在 s 到 t 的最短路上的边。

还是从 s 和 t 分别进行一次 BFS，记录距离分别为 ds 和 dt 。

对于一条有向边 (u, v) ，如果 $ds[u] + dt[v] + 1$ 等于 $ds[t]$ ，则说明必定存在经过 (u, v) 的最短路，否则一定不存在。

对于一条无向边，分别从两个方向上尝试。若均不成立，则不存在经过这条边的最短路，否则存在。

BFS - 应用

求长为偶数的最短路

给定一张无权图和起点 s 以及终点 t ，求 s 到 t 的最短的长度为偶数的途径（可以经过重复的点 and 边）。

将原图的每个点 u 拆为 $(u, 0)$ 和 $(u, 1)$ 两个点，建立新图。

若在原图上 u 向 v 连边，则在新图上让 $(u, 0)$ 向 $(v, 1)$ 连边， $(u, 1)$ 向 $(v, 0)$ 连边。

从 $(s, 0)$ 开始跑 BFS，求出到 $(t, 0)$ 的最短路即可。

BFS - 应用

求模某数意义下长度为某数的最短路

给定一张无权图和起点 s 以及终点 t ，求 s 到 t 的最短的长度对 k 取模余 r 的途径（可以经过重复的点 and 边）。

BFS - 应用

求模某数意义下长度为某数的最短路

给定一张无权图和起点 s 以及终点 t ，求 s 到 t 的最短的长度对 k 取模余 r 的途径（可以经过重复的点和边）。

类似的，将原图的每个点 u 拆为

$(u, 0), (u, 1), (u, 2), \dots, (u, k-1)$ 总共 k 个点，建立新图。

若在原图上 u 向 v 连边，则在新图上对所有的 $0 \leq i < k$ ，让 (u, i) 向 $(v, (i+1) \bmod k)$ 连边。

从 $(s, 0)$ 开始跑 BFS，求出到 (t, r) 的最短路即可。

时间复杂度 $O(k(n+m))$ 。

双端队列 BFS

双端队列 BFS, 顾名思义就是使用双端队列实现的一种特殊的 BFS。

什么是双端队列?

双端队列是一种可以在队首或队尾插入或删除元素的线性数据结构。

这里我们只需要用到：在队首插入一个元素，在队尾插入一个元素，以及删除队首元素。

双端队列 BFS 又称 0-1 BFS。顾名思义就是适用于边权为 0 或 1 的图上的 BFS。

双端队列 BFS

为维护 BFS 的性质，我们需要让队列保持递增且首尾差值不超过 1。

当拓展到一条边权为 0 的边时，将其插到队首。当拓展到一条边权为 1 的边时，将其插到队尾。

DFS - 原理

DFS (Depth First Search), 中文名是深度优先搜索。

该算法中, 如果从当前结点出发能直接抵达一个未访问过的结点, 则尝试访问该结点。如果从该点出发能直接抵达的所有结点都已被访问, 则回溯到上一个结点。

DFS - 实现

与搜索算法中的 DFS 类似的，通过递归的调用自身。

一个 DFS 的大致结构如下。

每次访问一个结点时，先给该点的 `vis` 值改为 `True`。

接着遍历所有相邻结点，如果该结点未被访问，则递归的访问，否则跳过。

遍历所有相邻结点后回溯。

DFS - 应用

DFS 序列是 DFS 调用过程中访问的结点编号序列。

在树上 DFS 得到的 DFS 序列满足每棵子树都对应 DFS 序列中连续的一段区间。

利用这一点，可以将子树问题变为序列上的区间问题，然后配合相关数据结构进行处理。

基本概念

拓扑排序是一种为有向无环图上的点排序的算法。
在一个有向无环图中，我们对图中的顶点以线性方式进行排序，使得对任意的有向边 (u, v) ，都有 u 在 v 前。

Kahn 算法

首先可以说明，有向无环图一定存在入度等于 0 的点。

考虑反证法，假设不存在入度为 0 的点，则对于一个点，总是可以选择一个点作为其前驱，从而构建一条任意长的途径。由于图中点数仅有 n 个，所以任意长度至少为 $n+1$ 的途径中总是存在一个点至少出现了两次，故总是可以找出一条回路，并进而找到一个环。这与有向无环图的定义矛盾，假设不成立。

Kahn 算法

考虑构建列表 L 。由上述论述，总是可以选出入度为 0 的点 u ，这个点对于任意剩余的有向边 (u, v) ，都有 u 在 v 前。故可以把 u 放入列表。

接下来构建新图 $G' = G[V/u]$ ，在新图上继续拓扑排序，直到点集为空。

Kahn 算法

实现时，首先遍历整张图，将入度为 0 的点放入集合 S 。

接下来，每次从 S 中任意取出一个点 u 放入列表 L ，然后将 u 的所有出边删除。对于边 (u, v) ，如果将该边删除后 v 的入度变为 0，则将 v 放入 S 中。

重复以上过程直至 S 为空，此时的列表 L 即为对原图拓扑排序的结果。

时间复杂度 $O(n + m)$ 。

应用

判断一张图是否是有向无环图

给定一张有向图，判断图中是否存在环。

尝试对图进行拓扑排序。

若算法结束时，构建的列表长度为 n ，则说明图中不存在环，否则说明图中存在环。

树的直径

直径是树上任意两结点之间的最长简单路径。

一棵树可以有多条直径。这些直径的长度是相同的。

可以使用两次 DFS 或树形 DP 的方式在 $O(n)$ 的时间复杂度下求出树的直径。

两次 DFS 求直径

定理：在一棵边权非负的树上，任意选择结点 y ，则距离 y 最远的结点 z 一定会是某条直径的一个端点。

那么可以首先任选一个点 x ，通过一次 DFS 找到距离 x 最远的点 y ，则 y 必然是某条直径的一个端点。从 y 出发再进行一次 DFS 找到离 z 最远的点，则 $\delta(y, z)$ 一定是一条直径。

严格意义上 DFS 只是手段，可以替换为任意的能求出树上离某个点最远的点的距离的算法，例如 BFS。

树形 DP 求直径

指定 1 号点为树根。考虑以 u 为根节点的子树内经过 u 的最长路径长度，则等于子树内以 u 为端点的最长链的长度与次长链的长度之和。对所有这样的和取最大值即为直径长度。

转移时，将自己的最长路径加上到父亲的边权后与父节点内存储的当前最长和次长路径长度比较即可。

树的重心

如果在树 T 中删去某个结点 v 及与其相连的边后，得到的图中每个连通分量的结点数量都不超过原数结点树的一半，则 v 被称为整棵树的重心。

树中所有节点到某个结点的距离和中，到重心 v 的距离最小。

树的重心如果不唯一，则恰有两个。这两个重心相邻。而且，删去它们的连边后，树将变为两个大小相同的连通分量。

把两棵树通过一条边相连得到一棵新的树，那么新树的重心在连接原来两棵树的重心的路径上。

DFS 求树的重心

通过 DFS 计算每个子树的大小。

在每个节点 u 记录以 u 为根的子树的大小。计算方式为，先递归的计算每个子树的大小，再将它们全部加起来，加上 1 即可。接着，先判断 u 的所有子树的大小是否不超过 $\frac{n}{2}$ 。如果均不超过，则判断除去整个以 u 为根的子树的连通分量的大小，即 n 减去以 u 为根的子树的大小，是否不超过 $\frac{n}{2}$ 。

若也满足，则 u 为树的重心。

时间复杂度： $O(n)$ 。

重心的应用

由于重心可以把树分为若干个大小不超过一半的连通分量，因此可以在树上进行分治算法，被称为点分治。
这一性质在一些交互题中也有所应用。

最小生成树，是无向连通图的子图，满足该子图包括所有点且连通，同时边权和为满足上述条件的图中最小的。

Kruskal 算法

Kruskal 算法是一种贪心算法。

将边按边权从小到大排序。构建一张子图，初始时包含原图中所有的点并且不包含任何边。

加入每条边时判断两个端点是否已经连通，如果是，舍弃这条边，否则将在图上连上这条边。

重复上述流程直至图连通。

Kruskal 算法的正确性证明

证明：任何时候 Kruskal 算法选择的边集都能被某棵 MST 包含。
算法刚开始是选择的边集为空集，显然成立。

假设某时刻 Kruskal 算法选择的边集能被某棵 MST 包含。记当前边集为 F ，被最小生成树 T 包含。考虑下一条加入的边 e 。

如果 $e \in T$ ，那么成立。

否则， $T + e$ 一定成环。由于 $F \cup \{e\}$ 不含环，则 $T + e$ 的环上一定存在不在 F 中的边。

假设一条这样的边是 f 。如果 $f < e$ ，则 f 应在 e 前选取，矛盾。
如果 $f > e$ ，则 $T - e + f$ 就比 T 更优了，与 T 是最小生成树不符。

故 $f = e$ ，因此 $T - e + f$ 包含 F 且也是最小生成树。归纳假设得证。

Kruskal 算法的实现

首先使用 `std::sort` 对边进行排序。在编写自定义比较器时，以边权为第一关键字。

接着使用并查集维护，初始时所有点在不同集合内。

每访问到一条边，先用并查集检查两端点是否在同一集合内。

如果是，直接跳过，否则答案加上边权并将两端点合并。

排序时间复杂度为 $O(m \log m)$ ，并查集时间复杂度不超过 $O(m \log n)$ ，总时间复杂度为 $O(m \log m)$ 。

Prim 算法

Prim 算法也是一种贪心算法。

Prim 算法中，会将点分入两个集合 A 和 B 中，初始时 A 包含一个任意结点， B 包含剩余结点。

重复 $n-1$ 次，每次选取边 $e = (u, v)$ 满足 $u \in A, v \in B$ 。如果有多条，取边权最小的一条。

接着将 e 选入构成最小生成树的边集，把 v 从 B 移除并加入 A 。

Prim 算法的正确性证明

证明：任何时候 Prim 算法选择的边集都能被某棵 MST 包含。

算法刚开始是选择的边集为空集，显然成立。

假设某时刻 Prim 算法选择的边集能被某棵 MST 包含。记当前边集为 F ，被最小生成树 T 包含。考虑下一条加入的边 e 。

如果 $e \in T$ ，那么成立。

否则， T 中至少包含一条 $f = (u, v)$ 满足 $u \in A$ 且 $v \in B$ 。

由于 e 为连接 A, B 的边中最小的一条，故 $e \leq f$ 。

$T - f$ 会使得图变为两个连通分量，而 e 又连接这两个分量，故 $T - f + e$ 也是生成树。

又因为 T 是最小生成树，所以 $T - f + e$ 不比 T 更优，故 $e = f$ 。
因此 $T - f + e$ 包含 F 且也是最小生成树。归纳假设得证。

Prim 算法的暴力实现

记录 `vis` 数组表示某个结点是否已在 A 集合中。

记录 `dis` 数组表示每个节点与 A 集合中点的连边中的最小值。

算法开始时, 将 `vis` 全部记为 `false`, `dis[1]` 记为某极小值, 其余的 `dis` 全部记为某极大值。

重复 n 次, 每次遍历整张图的 `vis` 为 `false` 的顶点, 找出 `dis` 值最小的点 u 。

将 `vis[u]` 改为 `true`, `dis[u]` 改为某极小值, 同时对于每条边 (u, v) , 设边权为 w , 则令 $\text{dis}[v] = \min(w, \text{dis}[v])$ 。

更新 `dis` 的值总共进行 $O(m)$ 次。每轮寻找最小值的点是 $O(n)$ 的, 因此总复杂度为 $O(n^2 + m)$ 。

Prim 算法的堆优化实现

考虑优化寻找最小值点的过程。

考虑用优先队列存储所有点的 dis ，即，将每个点作为二元组 $\{\text{dis}[u], u\}$ 插入优先队列。

每次寻找最小值点时，首先去除若干队首的存储在优先队列中的 dis 值大于存储在全局的 dis 值的无效元素。

然后再取出队首，即可得到最小值点。

由于进入优先队列的操作只有初始进队列和通过边拓展进队列两种，因此进出队列的操作总数是 $O(n + m)$ 的，故总时间复杂度为 $O((n + m) \log m)$ 。

输出方案

Kruskal 算法直接在每条边成功选入时存储即可。

Prim 算法如果需要输出方案时，需要在转移 dis 时额外标记每个点是通过哪条边转移过来的。

瓶颈生成树

无向图的瓶颈生成树是最大的边权值最小的生成树。

最小生成树一定是瓶颈生成树。因此，可以使用 Kruskal 算法或 Prim 算法求解。

最短路

s 到 t 的最短路，指的是带权图上 s 到 t 的所有途径中，权值最小的途径。

若 s 到 t 的路径上存在负环，则最短路不存在。下文若不特殊说明，总是认为不存在负环。

单源最短路问题，是给定一张图，求出 s 到其它所有点的最短路。

多源最短路问题，是给定一张图，求出所有点两两之间的最短路。

求解最短路问题有多种不同算法，适用条件与效率也互不相同。

Floyd 算法

Floyd 算法是一种全源最短路算法，用于求解任意两点间的最短路。

Floyd 算法本质上是动态规划。其核心思想为不断将 1 至 n 中的结点依次尝试加入路径的途径点。

Floyd 算法

具体的, 定义 $f(k, x, y)$ 表示只允许经过 (不包括 x 和 y) 编号在 1 至 k 之间的点的前提下, x 到 y 的最短路。

初始时, $f(0, x, y)$ 在 $x = y$ 时为 0, 在 x 到 y 直接有边相连时为这条边的边权, 否则为 ∞ 。

考虑转移。对于 $\forall 1 \leq n \leq k$, $f(k, x, y)$ 可以有两种来源:

- 第一种是不经过点 k , 这种情况下等于 $f(k-1, x, y)$ 。
- 第二种是经过点 k , 这种情况下也就是先从 x 走到 k , 再走到 y , 即 $f(k-1, x, k) + f(k-1, k, y)$ 。

两者取较小值即可。

Floyd 算法

实际上, 在实现中, 第一维是可以压缩的。

当更新 $f(k, x, y)$ 时, 如果 $f(k-1, x, k)$ 和 $f(k-1, k, y)$ 的值分别等于 $f(k, x, k)$ 和 $f(k, k, y)$, 那么直接去掉第一维就是可行的。

对于前者, 在更新 $f(k, x, k)$ 时, 转移路径一就是 $f(k-1, x, k)$, 而转移路径二根据公式有 $f(k-1, x, k) + f(k-1, k, k)$ 。

由于不存在负环, 故 $f(k-1, k, k) \geq 0$ 。又因为 $f(0, k, k) = 0$, 而 $f(k-1, k, k) \leq f(0, k, k)$, 故 $f(k-1, k, k) = 0$ 。

因此, 两条转移路径得到的结果都是 $f(k-1, x, k)$, 故它的值不会改变。后者的证明是同理的。

因此, 可以将转移式直接写为

$$f(x, y) = \min(f(x, y), f(x, k) + f(k, y))。$$

Floyd 算法

显然 Floyd 算法的时间复杂度是 $O(n^3)$ ，空间复杂度是 $O(n^2)$ 。
Floyd 算法可以对任意不含负环的图使用，复杂度较高，但易于实现。

Floyd 算法的应用

最小环

给定正权有向图，求权值和最小的环。

考虑环上编号最大的点 u 。

这个环可以拆解为 $f(u-1, x, y), (y, u), (u, x)$ 三段。

因此，当外层循环到 u 时，在更新最短路之前先枚举所有的 x 和 y ，然后计算 $dis(x, y) + w(y, u) + w(u, x)$ 即可。

Floyd 算法的应用

传递闭包

给定有向图，判断任意两点是否连通。

Floyd 算法的应用

传递闭包

给定有向图，判断任意两点是否连通。

只需要把 Floyd 算法的边权改为 0 和 1，然后把加法改为与运算，把取最小值改为或运算即可。

Bellman-Ford 算法

Bellman-Ford 算法是一种单源最短路算法。
该算法是一种基于松弛操作的最短路算法，对任意图适用，且能判断不存在最短路的情况。

Bellman-Ford 算法

松弛是一种重新计算距离估计值的操作。

设 $dis(u)$ 表示源点到某个结点的距离估计值。对于边 (u, v) ，松弛操作就是 $dis(v) \leftarrow \min(dis(v), dis(u) + w(u, v))$ 。

Bellman-Ford 算法所做的就是不断尝试对每一条边进行松弛，直到没有边可以被松弛。

在最短路存在的情况下，每轮松弛会使得最短路的边数至少 $+1$ ，而最短路的边数最多为 $n - 1$ ，故整个算法最多执行 $n - 1$ 轮松弛操作，故时间复杂度为 $O(nm)$ 。

Bellman-Ford 算法

若从源点出发会抵达一个负环，则松弛操作会无休止的进行下去。

由前面的结论，如果第 n 轮循环时还存在能松弛的边，则说明从 s 出发可以抵达一个负环。

注意该方法只能判断从某个点出发能不能抵达一个负环。如果要判断整张图是否存在负环，常见的做法是建立超级源点，向每个点连一条边，然后从超级源点开始做 Bellman-Ford。

关于 SPFA

- 它死了。

关于 SPFA

SPFA 是 Bellman-Ford 的队列优化版本。

很显然只有上一次被松弛的点所连接的边才能引起下一次的松弛。

使用队列维护可能引起松弛操作的点。具体的，每次从队首取出点，对其出边尝试松弛。

松弛成功后，若到达的点不在队列中，则将其加入队列。

关于 SPFA

SPFA 的期望复杂度为 $O(m)$ ，在随机图上表现优秀。

但 SPFA 的最坏时间复杂度为 $O(nm)$ ，可以通过随机网格图等方式容易取到上界。

如果图中存在负权边且无特殊性质，则 SPFA 仅在 Bellman-Ford 算法可以通过的情况下可能称为标算的一部分。

Dijkstra 算法

Dijkstra 算法是一种求解非负权图上单源最短路径的高效算法。
这是一种基于贪心的算法。

Dijkstra 算法

将结点划分为两个集合 S 和 T ，其中源点 s 到 S 集合中点的最短路已确定，到 T 集合中点的最短路未确定。

初始化 $dis(s) = 0$ ，其余点的 dis 值为 ∞ 。初始时所有点都在集合 T 中。

重复 n 次操作：

- ① 从 T 集合中取出一个最短路估计值最小的点放入 S 集合。
- ② 对刚刚操作的点的所有出边进行松弛操作。

直至 T 集合为空。

Dijkstra 算法的正确性

证明：每次取出结点时，取出的点 u 满足 $D(u) = dis(u)$ 。
其中 $D(u)$ 为 s 到 u 的最短路实际长度， $dis(u)$ 为估计长度。
初始时 $S = \emptyset$ ，假设成立。

Dijkstra 算法的正确性

考虑反证。设 u 为第一个在加入 S 时不满足 $D(u) = \text{dis}(u)$ 的点。

由于 s 总是第一个加入 S ，且满足 $D(s) = \text{dis}(s)$ ，故 u 加入 S 前 S 不为空集。

若不存在 s 到 u 的路径，则 $D(u) = \text{dis}(u) = \infty$ ，与假设矛盾。

Dijkstra 算法的正确性

记 s 到 u 的最短路径为 $s \rightarrow x \rightarrow y \rightarrow u$, 其中 y 为路径上第一个位于 T 集合中的点, x 为其前驱。

由于在 u 之前取出的点都满足 $D = dis$, 故 $D(x) = dis(x)$ 。

此时对 y 进行松弛, 得到 $D(y) = dis(y)$ 。

Dijkstra 算法的正确性

由于 $s \rightarrow x \rightarrow y \rightarrow u$ 边权非负，故

$$\text{dis}(y) = D(y) \leq D(u) \leq \text{dis}(u).$$

但由于 y 仍在 T 集合内，故 $\text{dis}(u) \leq \text{dis}(y)$ ，故有

$$D(u) = \text{dis}(u) = \text{dis}(y), \text{ 与假设矛盾。}$$

综上，每次取出结点时，取出的点 u 满足 $D(u) = \text{dis}(u)$ 。

Dijkstra 算法的实现

朴素的实现方式为每次寻找 dis 最小的点时直接枚举。由于总共寻找 n 轮，而每条边松弛 1 次，故总时间复杂度 $O(n^2 + m)$ 。

可以使用堆进行优化。使用优先队列维护点及其距离，以距离为第一关键字排序。

弹出结点时，若该结点已被松弛过，则将其跳过。时间复杂度 $O(m \log n)$ 。

稠密图上使用朴素实现会比堆优化实现更优。

分层图

例题

给定一张有向无环图，现从 s 到 t 的路径上可以将至多 k 条边的边权设为 0，求最短路边权和。

将图拆为 $k+1$ 层，第 i 层表示使用了 i 次将边权改为 0。

若原图存在 $u \rightarrow v$ 的边，则新图上每层的 u 到 v 正常连边，每层的 u 往下一层的 v 连一条权值为 0 的边。

从 s 开始跑单源最短路，记录到每层 t 的最短路的最小值即可。

最近公共祖先

有根树上的点 w 为点 u 和点 v 的公共祖先，当且仅当 w 既是 u 的祖先，又是 v 的祖先。

w 为 u 和 v 的最近公共祖先 (LCA)，当且仅当 w 为 u 和 v 的最近公共祖先中深度最深的点。

最近公共祖先的基本性质

- 可重复贡献: $\text{LCA}(u, u) = u$ 。
- 交换律: $\text{LCA}(u, v) = \text{LCA}(v, u)$ 。
- 结合律: $\text{LCA}(\text{LCA}(u, v), w) = \text{LCA}(u, \text{LCA}(v, w))$ 。
- u 是 v 的祖先, 当且仅当 $\text{LCA}(u, v) = u$ 。
- $\text{dis}(u, v) = h(u) + h(v) - 2h(\text{LCA}(u, v))$, 其中 dis 为距离, h 为深度。

最近公共祖先的朴素算法

首先找到更深的一个点，让该点重复向上跳至父亲直到两点深度相同。

接着让两点同时重复向上跳至父亲，直至两点相遇。相遇位置即为最近公共祖先。

单次查询时间复杂度 $O(n)$ 。

最近公共祖先的倍增算法

上述算法总共包括两步：调整两点深度至相同位置，然后同步向上跳至公共祖先。
分步进行优化。

最近公共祖先的倍增算法

我们称点 u 到根节点路径上的第 $k+1$ 个点为 u 的 k 级祖先。
第一步的优化关键在于给定一个 k ，需要快速的找到 u 的 k 级祖先。

考虑二进制拆分。 k 总是可以被表示为若干个 2 的不同幂次的和。

考虑记录每个点的 2^0 级祖先， 2^1 级祖先， 2^2 级祖先等结点。
记 $f(i, u)$ 为 u 的 2^i 级祖先，其中 $2^0 = 1$ 级祖先为 u 的父节点。
不难发现， $f(i, u) = f(i-1, f(i-1, u))$ 。这一步的时间复杂度为 $O(n \log n)$ 。

最近公共祖先的倍增算法

当要让 u 跳跃至 k 级祖先时，首先将其进行二进制拆分为

$$\sum_{i=0} w_i \cdot 2^i。$$

其中， w_i 总为 0 或 1。

对于每个为 1 的 w_i ，令 $u \leftarrow f(i, u)$ 。

这一步的时间复杂度为 $O(\log n)$ 。

最近公共祖先的倍增算法

当两点深度相同时，若两点位于同一位置，则直接返回该点为最近公共祖先即可。

否则，我们希望通过让 u, v 向上跳跃，使得 $u \neq v$ ，且

$$f(0, u) = f(0, v)。$$

假定此时 u 的当前深度和目标深度的差值为 k ，则 k 一定存在唯一的二进制拆分。

一种找出其二进制拆分的贪心算法为：从大到小尝试每个 2^i 。

若 $k < 2^i$ ，则忽略，否则令 $k \leftarrow k - 2^i$ 。

最近公共祖先的倍增算法

这里采用类似的策略。从 $\lfloor \log_2 h(u) \rfloor$ 到 0 枚举 i 。

对于每个枚举到的 i ，若 $f(i, u) = f(i, v)$ ，说明 $k < 2^i$ ，此时应忽略 i 。

否则，令 $u \leftarrow f(i, u)$ ， $v \leftarrow f(i, v)$ ，表示令 u 和 v 均向上跳 2^i 层。

枚举完毕后，返回 $f(0, u)$ 即可。

这一步的时间复杂度还是 $O(\log n)$ 。

故预处理复杂度为 $O(n \log n)$ ，单次查询复杂度为 $O(\log n)$ 。

最近公共祖先的应用

树上路径边权极值

给定带权树，共 Q 次询问，每次询问树上两不同点之间的简单路径上边权最大的边的权值。

最近公共祖先的应用

树上路径边权极值

给定带权树，共 Q 次询问，每次询问树上两点之间的简单路径上边权最大的边的权值。

注意到，最大值本身也是可以倍增的。

记 $g(i, u)$ 表示点 u 到点 $f(i, u)$ 的路径上边权的最大值。

容易发现， $g(i, u) = \max(g(i-1, u), g(i-1, f(i-1, u)))$ 。

最近公共祖先的应用

树上路径边权极值

给定带权树，共 Q 次询问，每次询问树上两点之间的简单路径上边权最大的边的权值。

在求解 LCA 的过程中维护 M 表示已跳跃部分最大值的边权最大值。

每次 $u \leftarrow f(i, u)$ 时，令 $M \leftarrow \max(M, g(i, u))$ 。对 v 同理。

若最终 $u \neq v$ ，则我们实际求出的 LCA 是 $f(0, u)$ 而非 u 。

此时应记得令 $M \leftarrow \max(M, g(0, u), g(0, v))$ 。

写在后面

图论是一个繁杂的版块，相关知识点非常多，这里限于时间不能讲的面面俱到。

与有向图连通性相关的各类 Tarjan 算法，图匹配问题，树上的各类算法，网络流，还有各类杂项算法尽管本节课内没有提及，但都是图论中相当重要的算法。

本人能力有限，可能对知识点讲的不够充分。学有余力的同学可以参考其它的相关资料。

也许会有用.....

- OI Wiki: <https://oi-wiki.org>
- 一份相关的题单: <https://www.luogu.com.cn/training/4969>

感谢聆听!